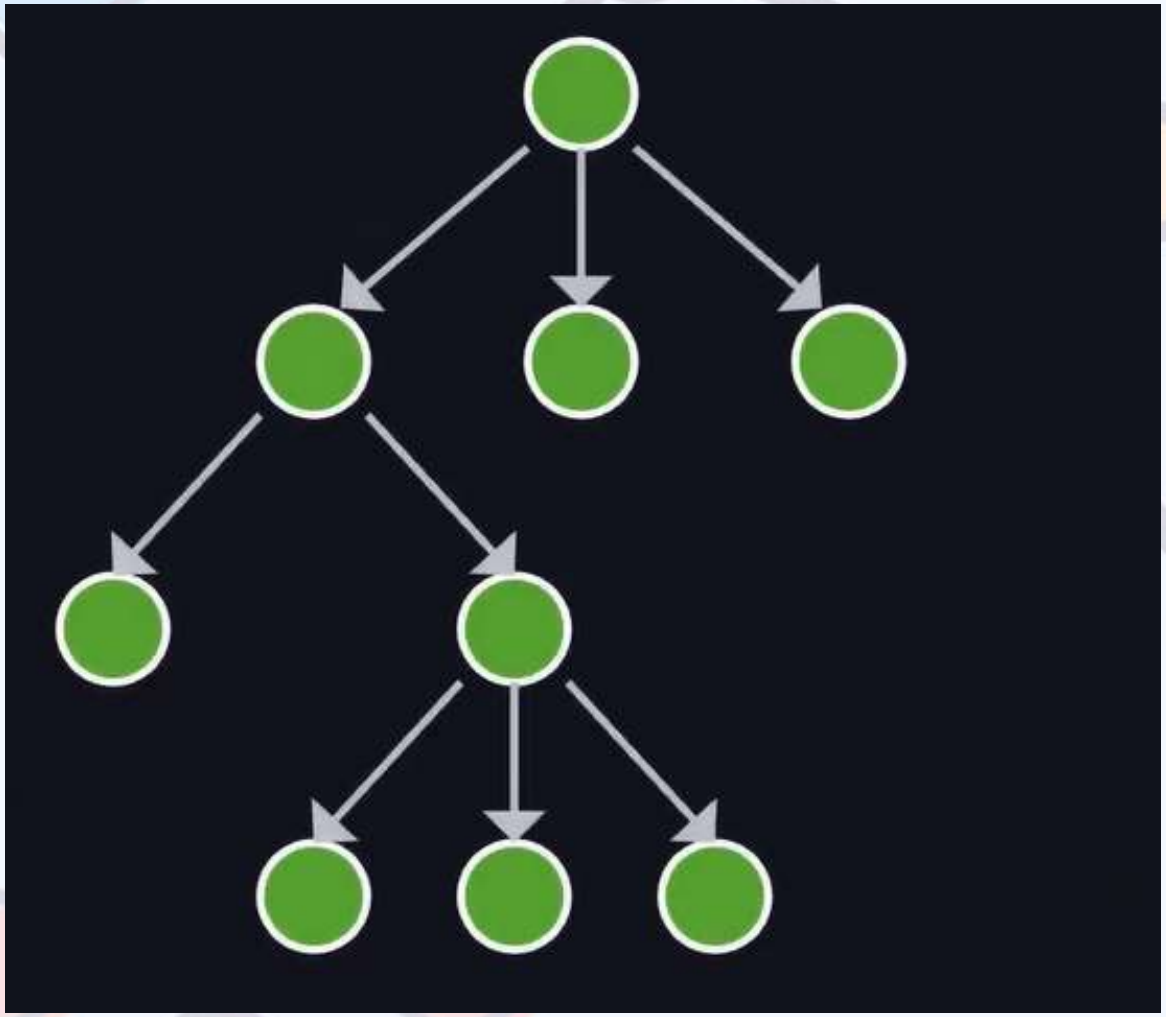
LECTURE CONTINUED

LECTURE NO 21

Binary Search Trees - Extended

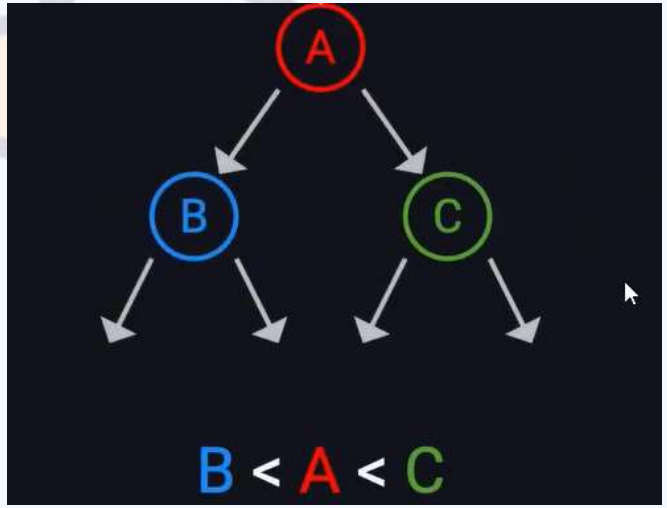
SQUADRON LEADER (RETD) ABDUL REHMAN

TREE



BINARY SEARCH TREES

- A **binary search tree** is a binary tree that:
- Maintains a specific order among its elements to facilitate
  - Efficient search
  - Insertion
  - Deletion
- Each node in a BST follows
  - Left child’s value is less than its own value
  - Right child’s value is greater than or equal to its own

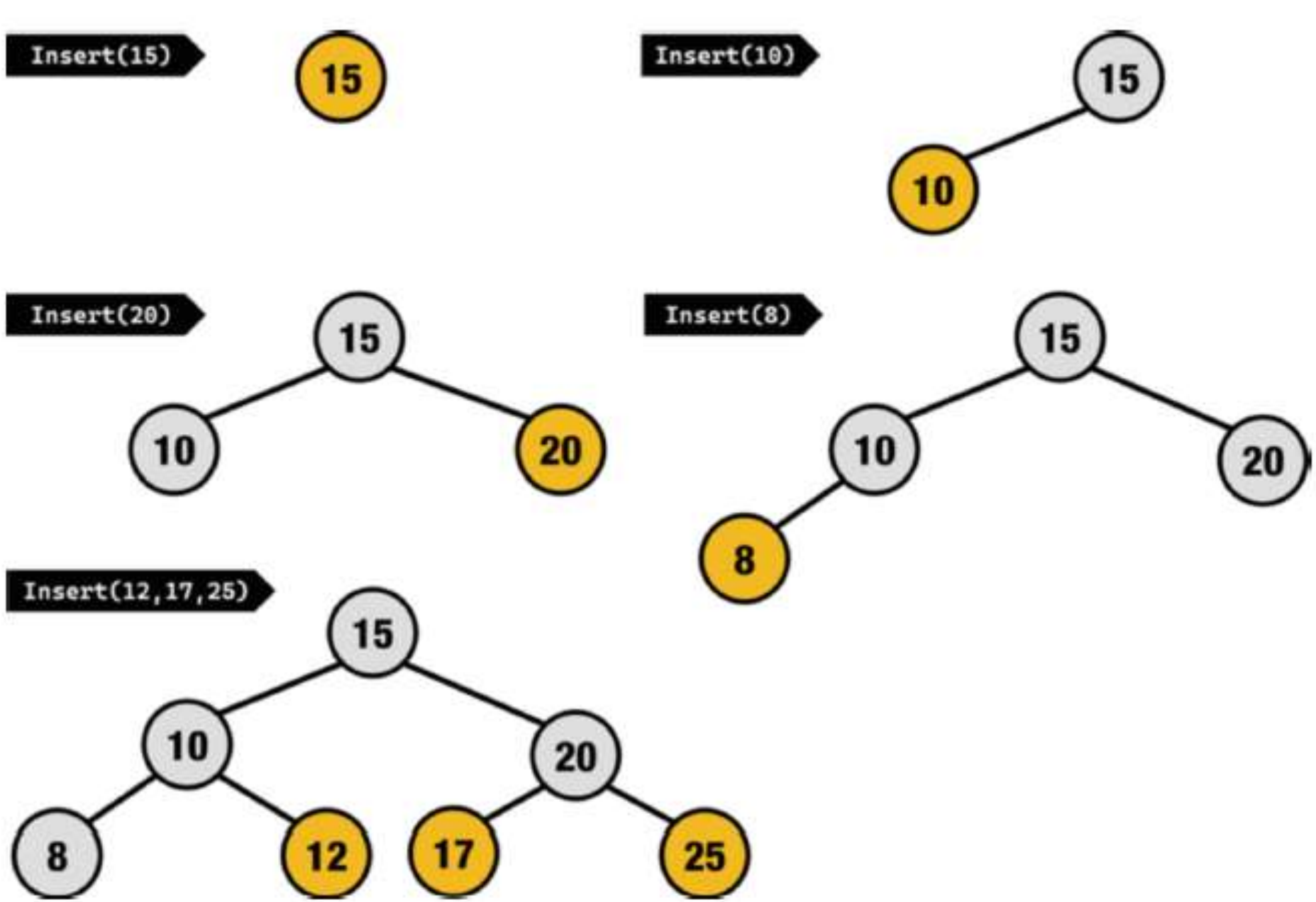


PROPERTIES OF BST

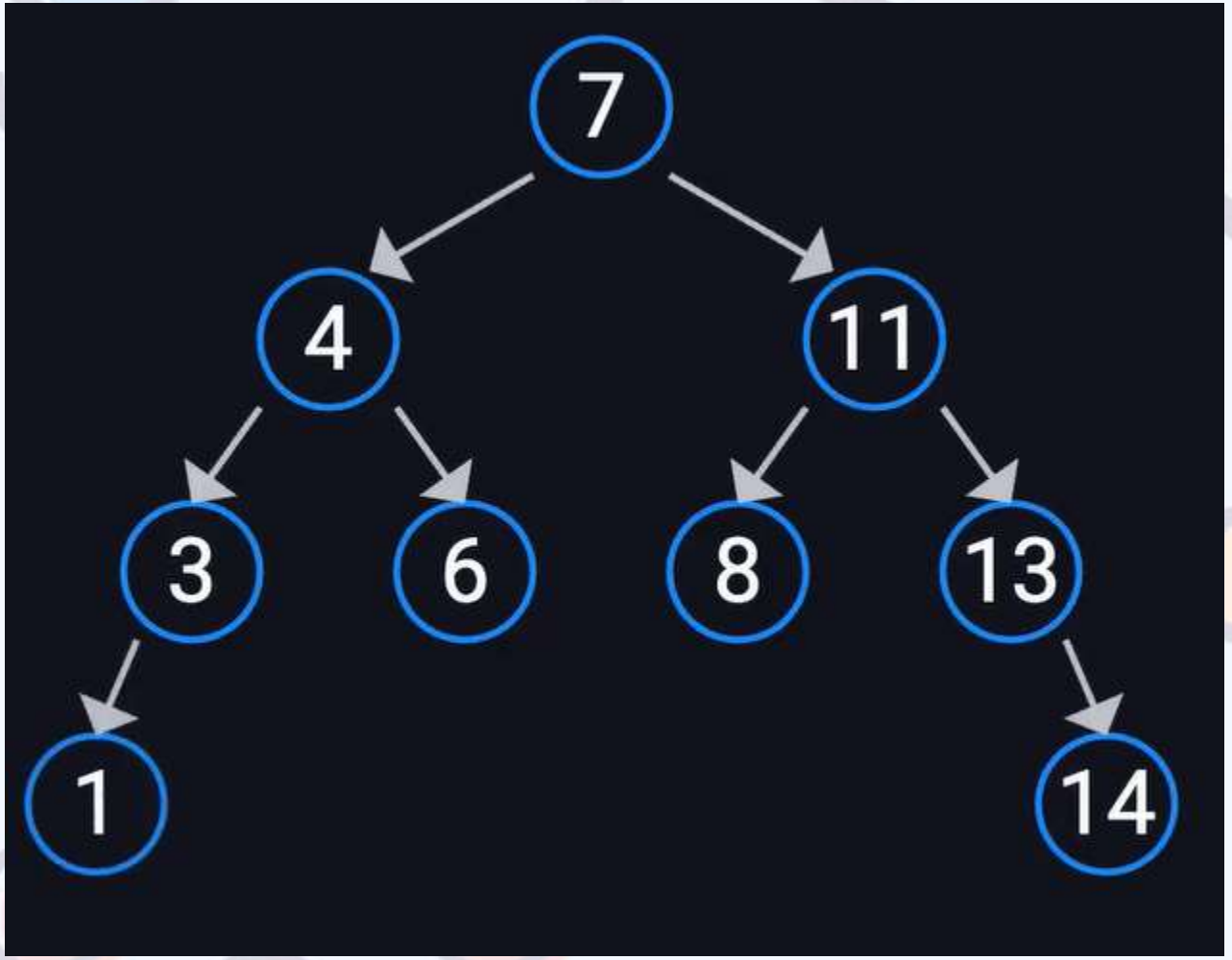
- **Ordering Property:** For any node in a BST, all keys in the left subtree are less than the key of node, and all keys in the right subtree are greater than or equal to the node
- **Unique Path:** There is a unique path from the root to any other node.
- **Efficient Search:** The ordering property allows for efficient searching, with average and worst-case time complexities of $O(\log n)$ and $O(n)$, respectively, depending on the tree's balance.

BASIC OPERATIONS IN BST

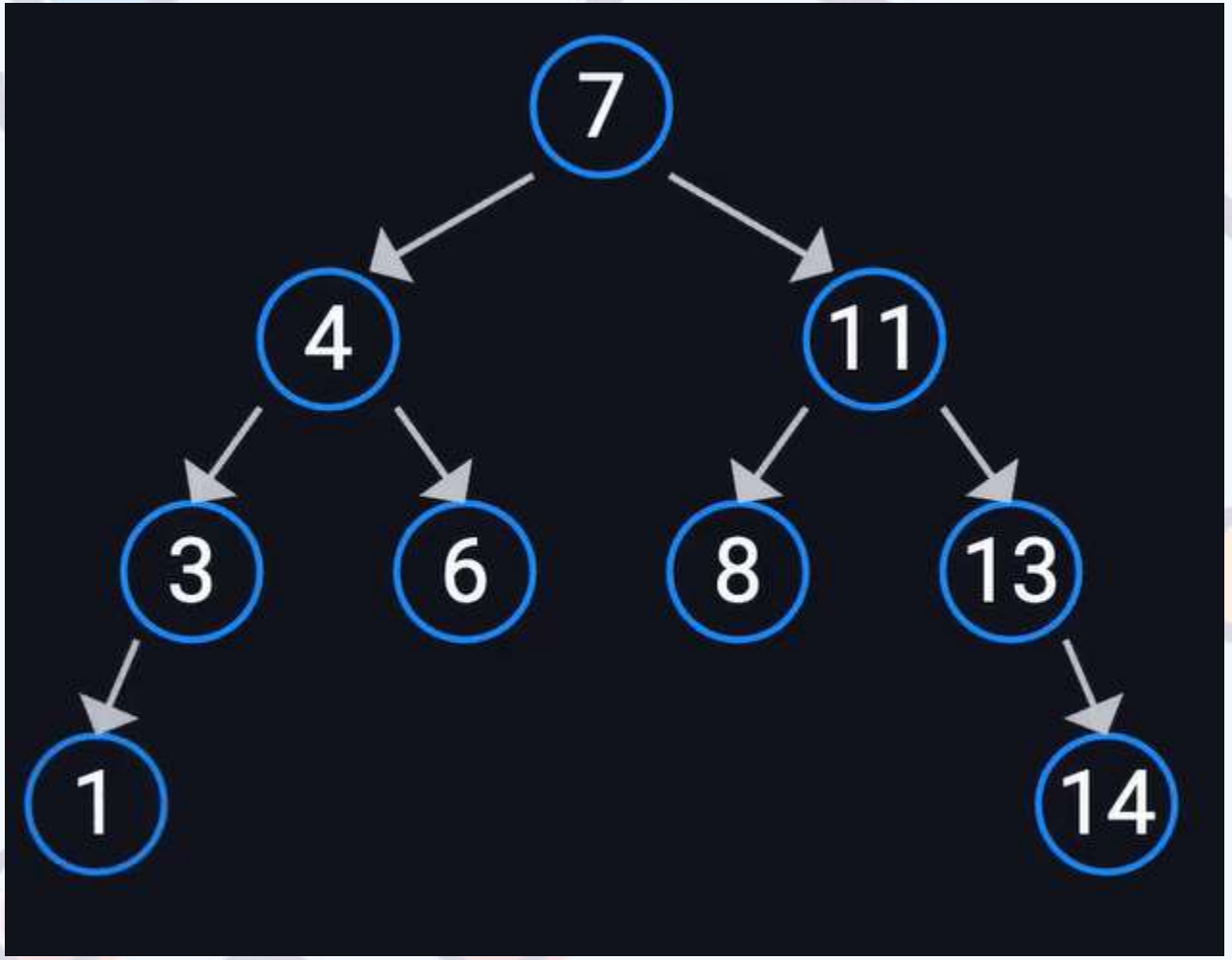
- **Insertion:** Inserts new elements while maintaining the BST's ordered property. The insertion finds the correct position for the new node to keep the tree sorted.
- **Deletion:** Handles three scenarios: deleting a leaf node, a node with a single child, or a node with two children. Each case is managed to preserve the BST's ordered structure.
- **Searching:** Involves comparing the target value with the nodes, starting from the root and traversing the tree accordingly. This process repeats until the target is found or the search reaches a leaf node.


Insert(15)
15
Insert(10)
15
10
Insert(20)
15
10
20
Insert(8)
15
10
20
8
Insert(12,17,25)
15
10
20
8
12
17
25

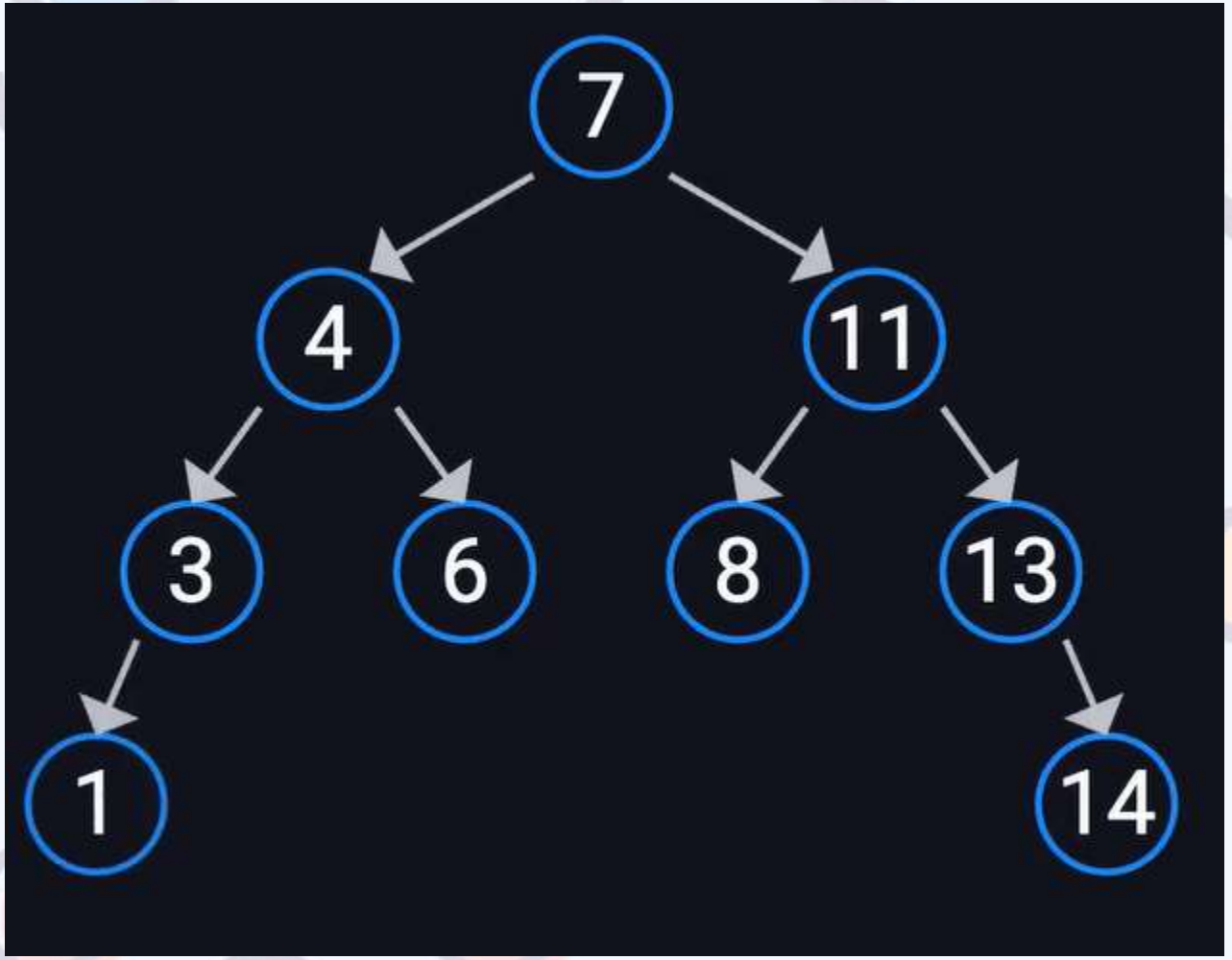
INSERT 7, 4, 11, 6, 3, 8, 13, 14, 1



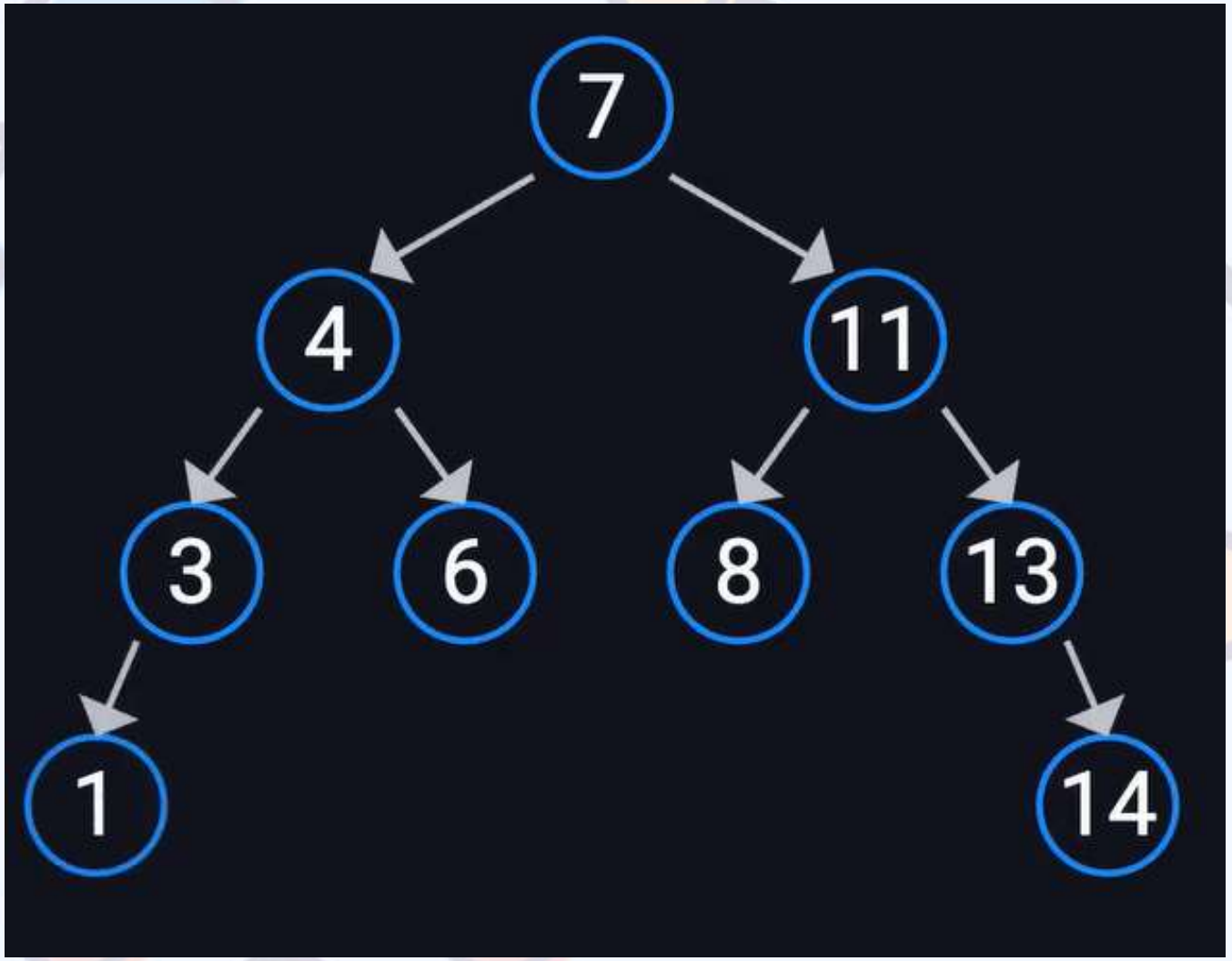
NOW SEARCH FOR ELEMENT 6



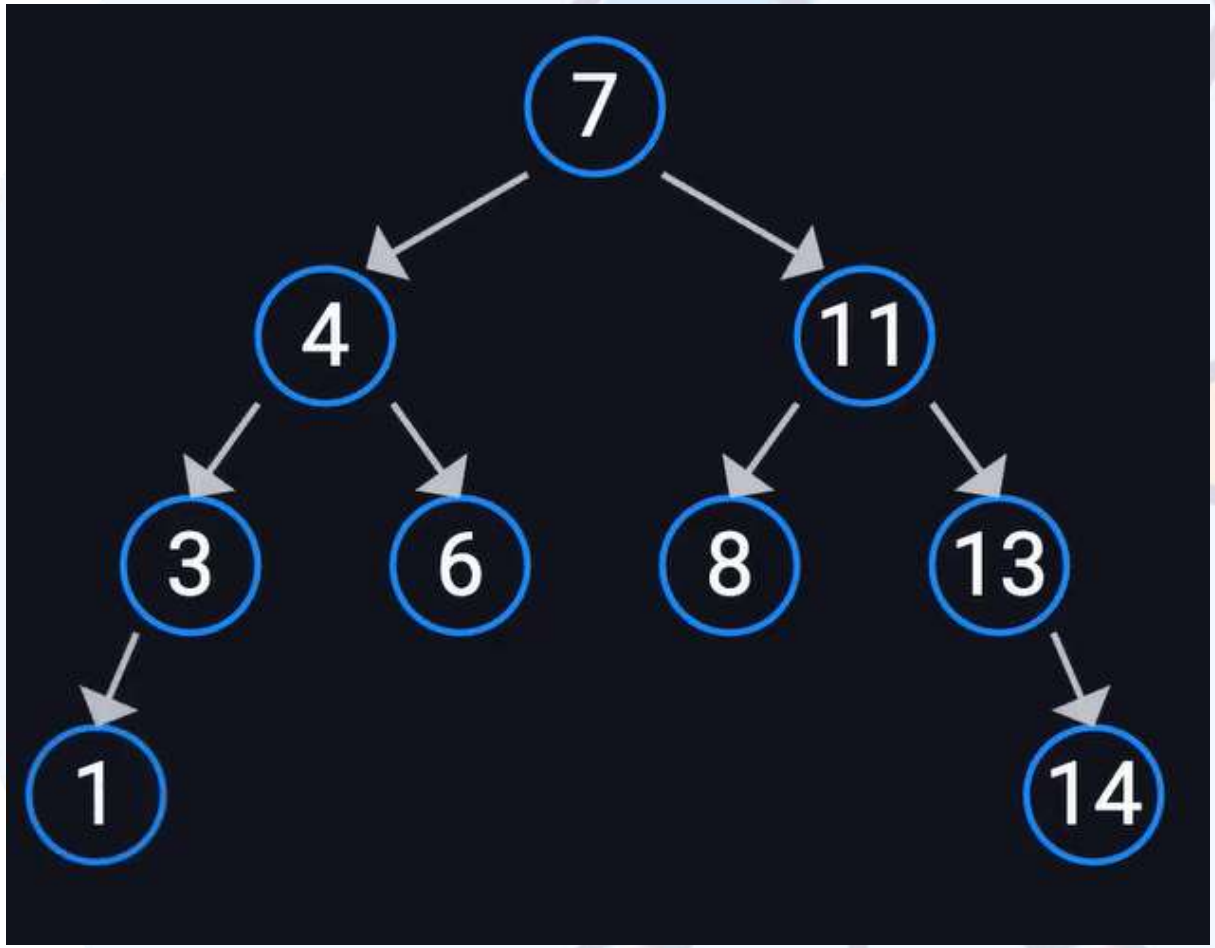
NOW SEARCH FOR ELEMENT 12



DELETE 14

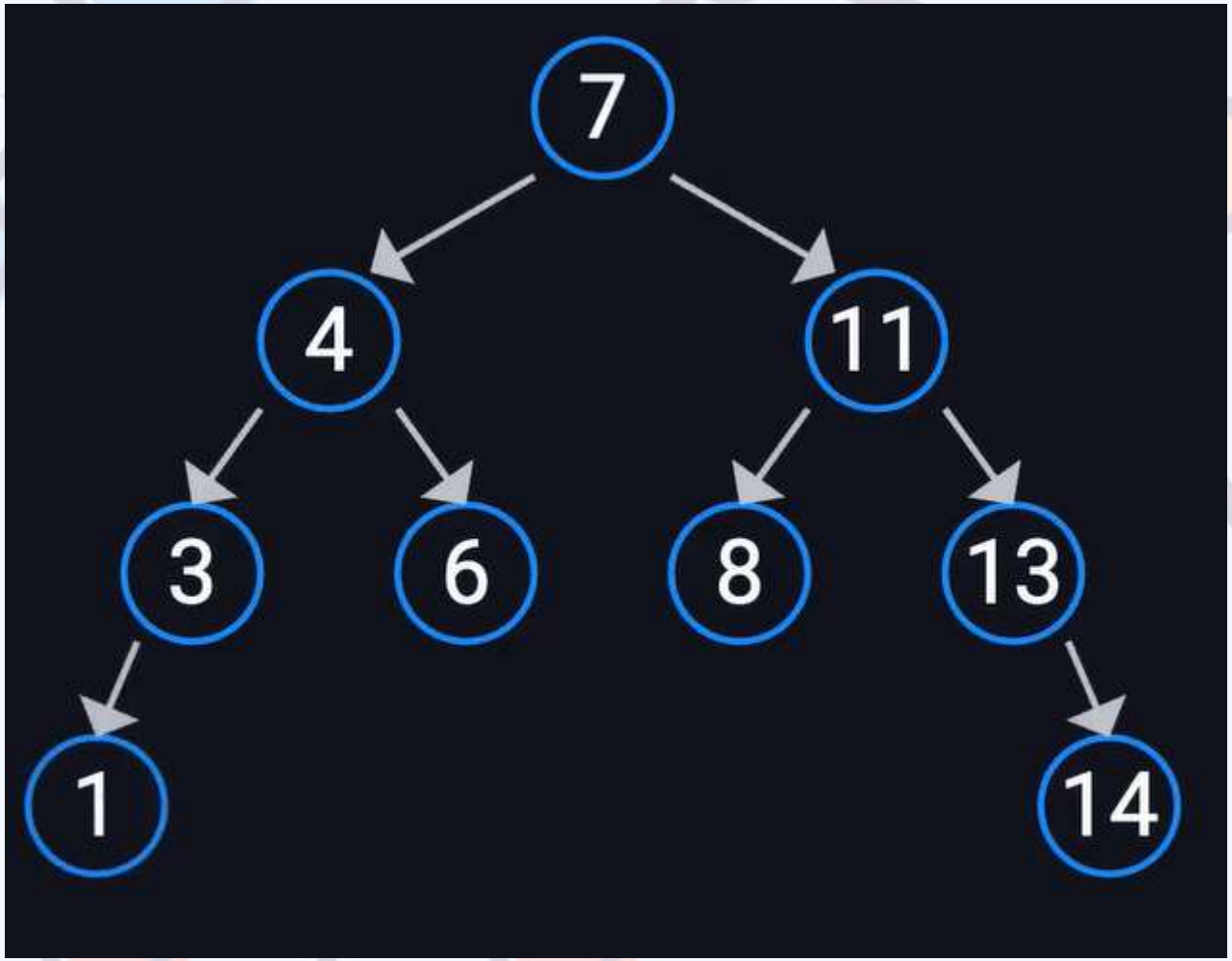


DELETE 14

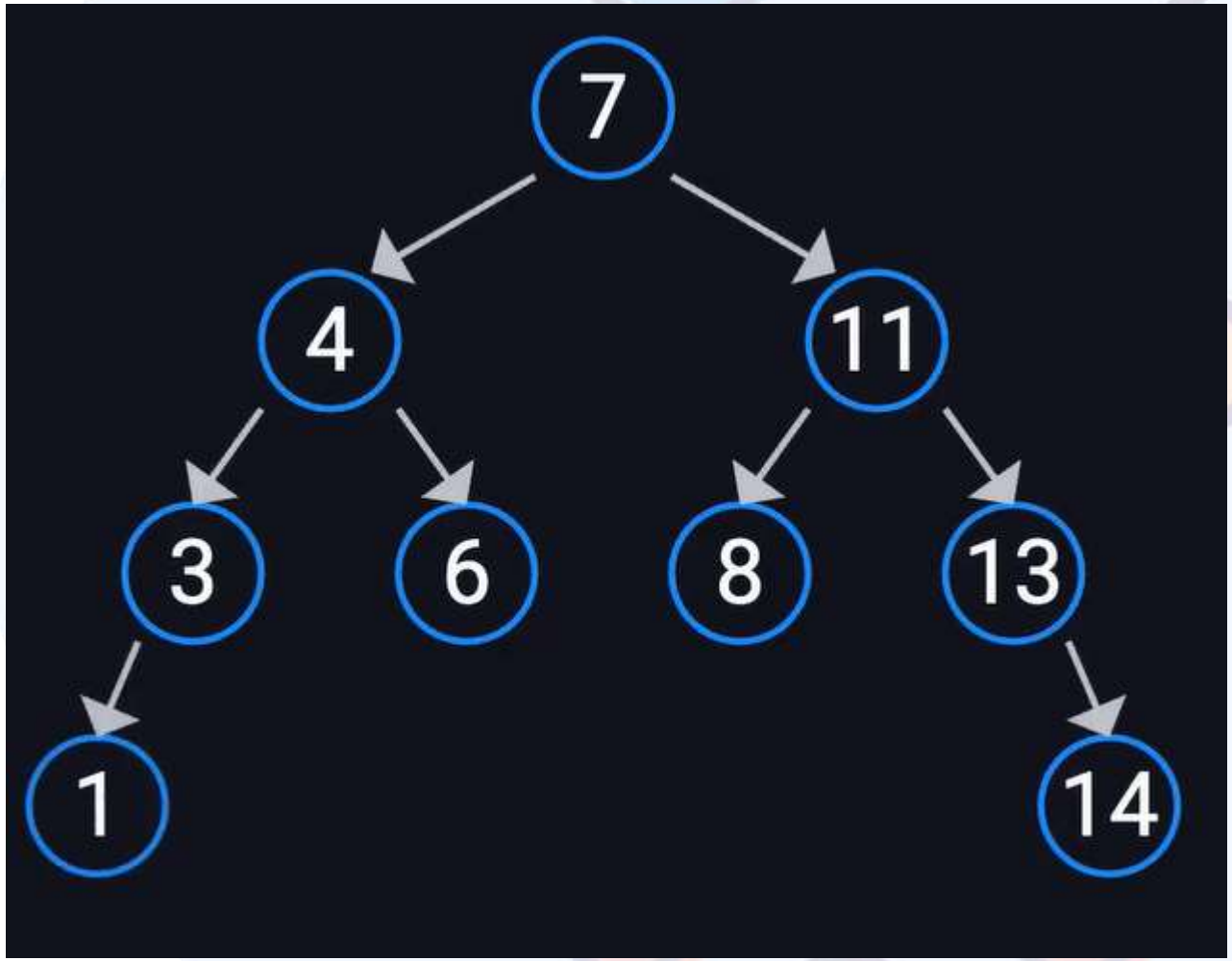



```
if (curr == curr->parent->right){
    curr->parent->right = nullptr;
    delete curr;
}
else{
    curr->parent->left = nullptr;
    delete curr;
}
```

DELETE 13 OR DELETE 3

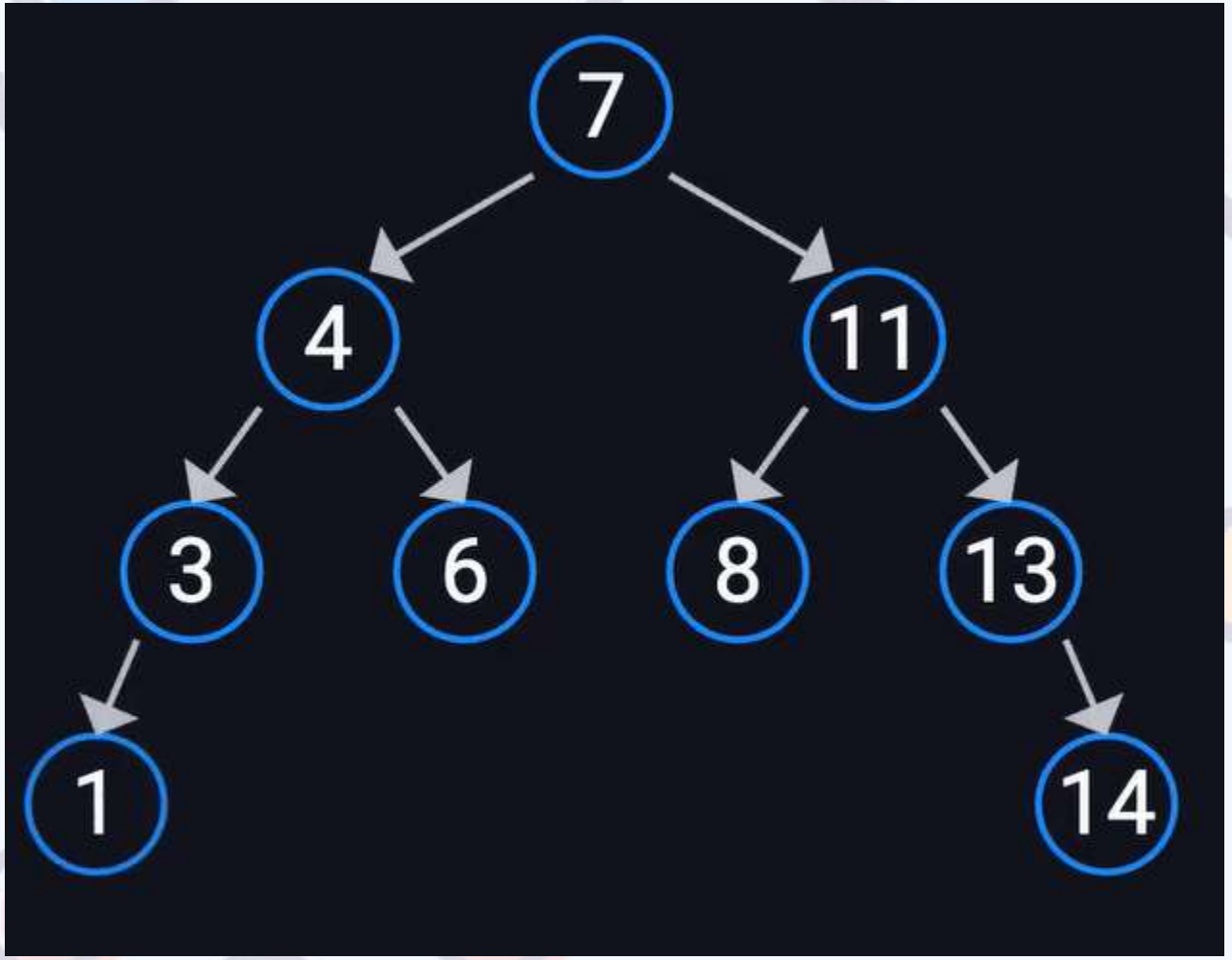


DELETE 13 OR DELETE 3



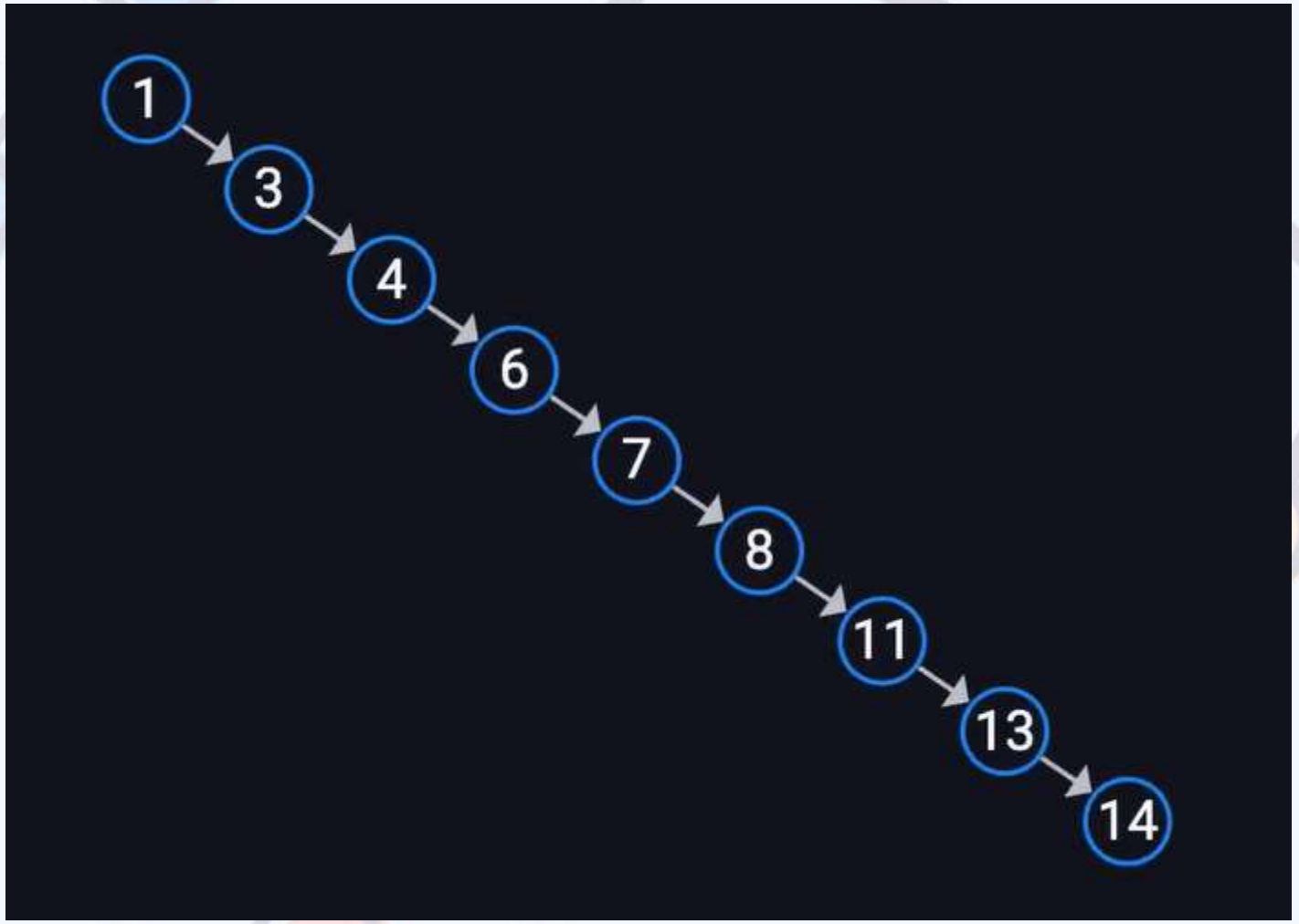

```
if(curr->left == nullptr){
    curr->right->parent = curr->parent;
    if(curr == curr->parent->right)
        curr->parent->right = curr->right;
    else
        curr->parent->left = curr->right;
    delete curr;
}
else if(curr->right == nullptr){
    curr->left->parent = curr->parent;
    if(curr == curr->parent->right)
        curr->parent->right = curr->left;
    else
        curr->parent->left = curr->left;
    delete curr;
}
```

NOW TRY DELETING 11 OR 4 OR 7

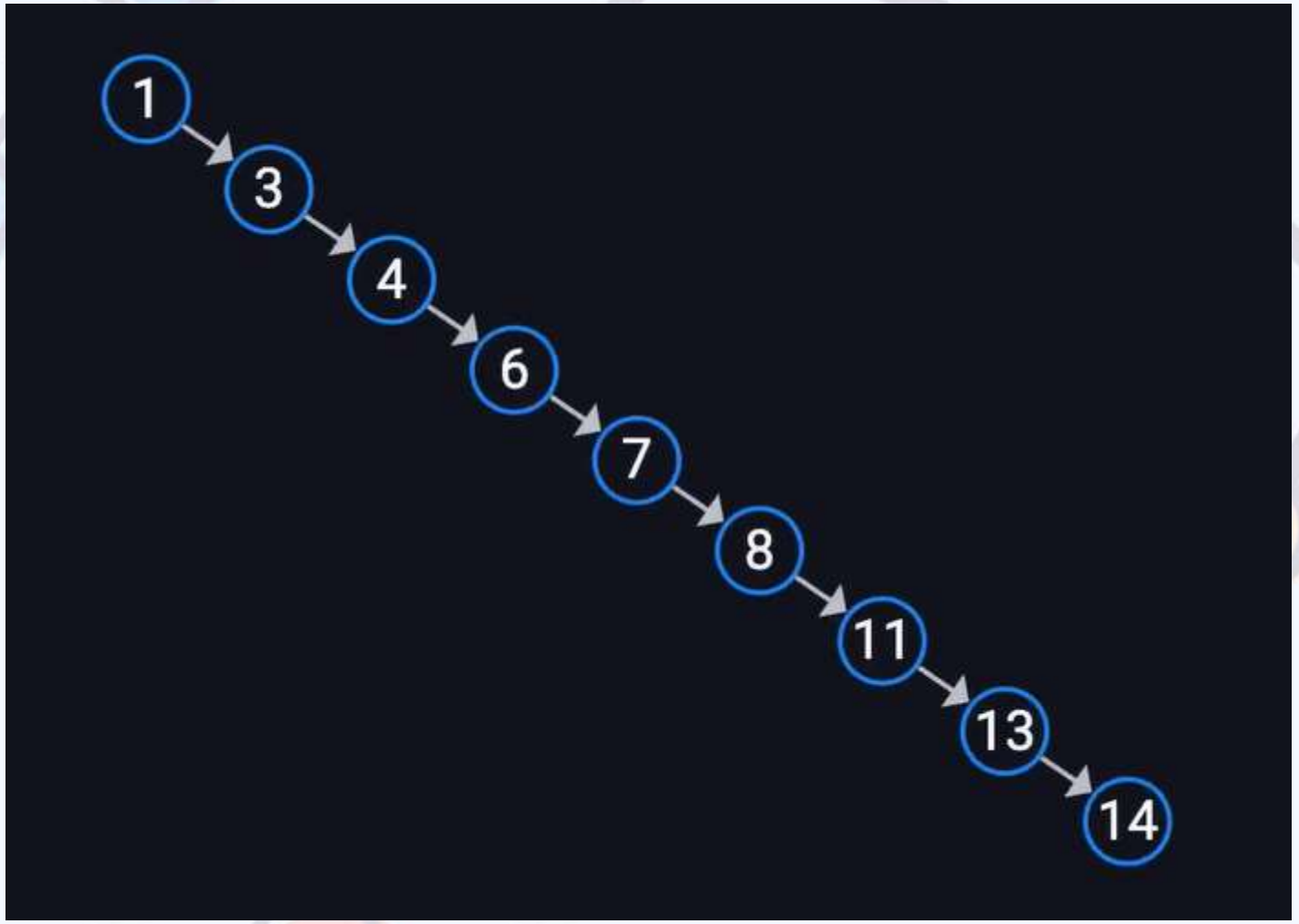


INSERT 1, 3, 4, 6, 7, 8, 11, 13, 14

NOW SEARCH ELEMENT 8



NOW SEARCH ELEMENT 17



SOLUTION?
SELF BALANCING TREES

- AVL
- Red Black Tree
- Splay
- B Tree
- B+ Tree
- Treap

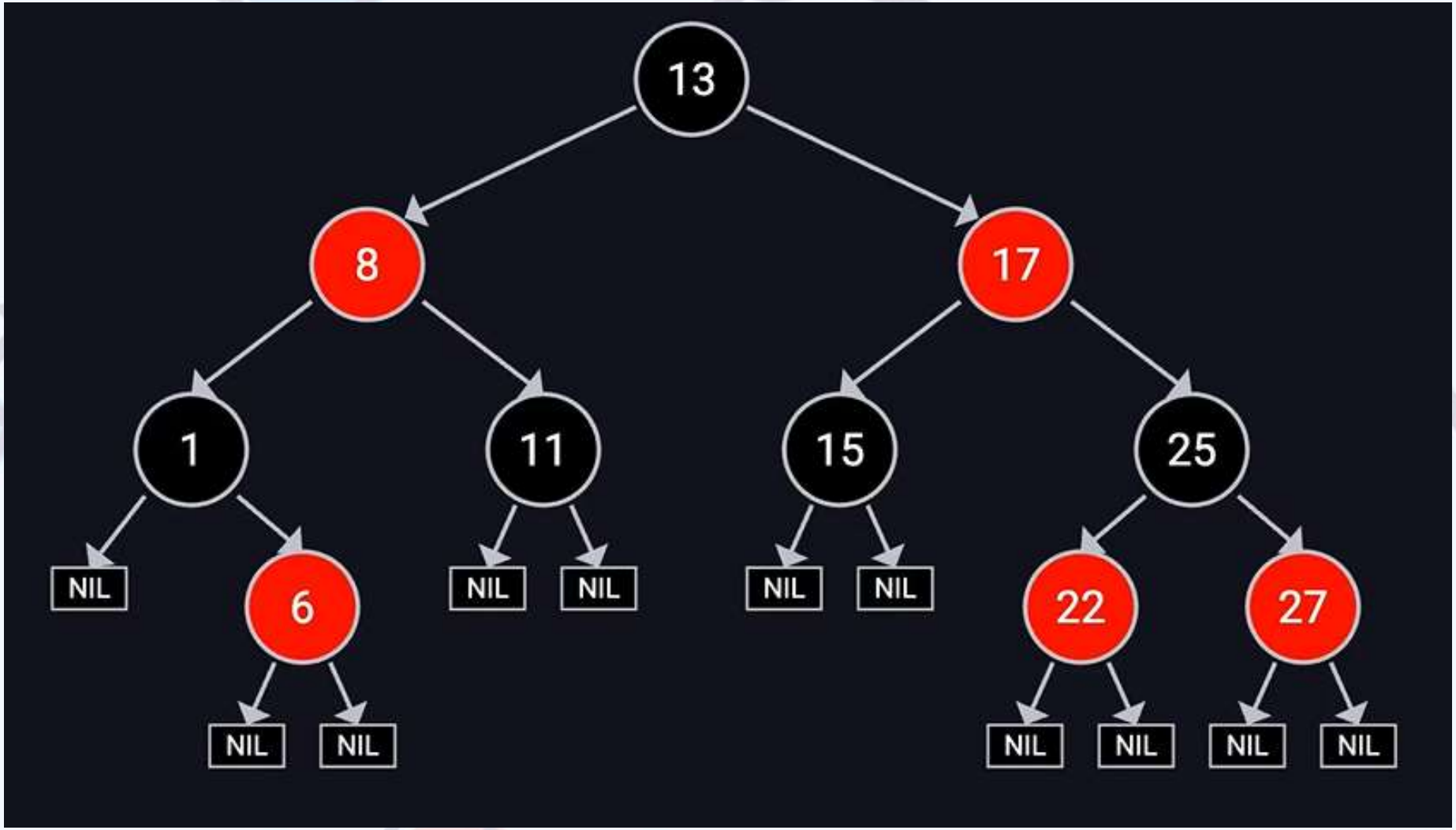
AVL (ADELSON-VELSKY AND LANDIS

- First self-balancing BST ever invented
  - It is the **Strictest** of the variants.
- For every node, the difference in height between its left and right subtrees (the Balance Factor) can be no more than 1, 0, or -1.
- If insertion or deletion causes Balance Factor to hit 2 or -2
  - The tree performs Rotations
- **In the Field:**,
  - It is faster for Lookups
  - Insertions/ Deletions : slower as they trigger frequent rotations

RED BLACK TREE

- They are **Less Strict than AVL Trees**
- **The Rules:**
  - Each node is either Red or Black
  - The root is always Black
  - Red nodes cannot have Red children
  - All leaf Nodes (NIL Nodes) are black
  - Every path from a node to its descendant NIL nodes must have the same number of Black nodes
- **The Mechanism:** It uses "color flips" and rotations to ensure that the longest path from the root to a leaf is no more than twice as long as the shortest path
- **In the Field:** This is the best **ALL-ROUNDER** for general purpose databases and language libraries.

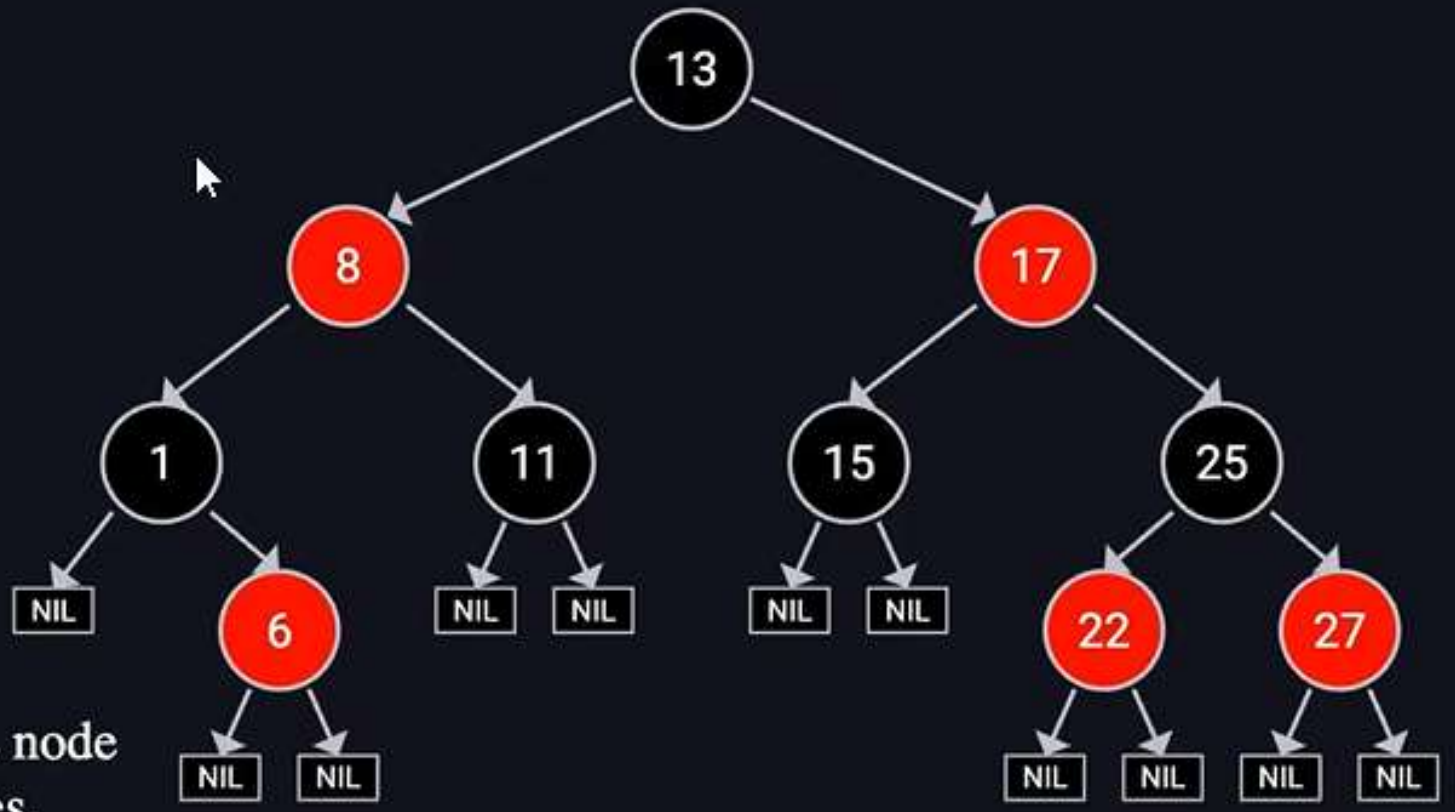
RED BLACK TREE



RED BLACK TREE

5 Rules of Red-Black Tree

1. Every node is either red or black
2. Root node is black
3. All Leaf(NIL) nodes are black
4. If a node is red, then both its children are black
5. For each node, every path to a NIL node has the same number of black nodes.



SOLUTIONS

VARIANT	BALANCING STRATEGY	BEST CASE USE
AVL	Strict height difference (≤1)	Fast lookups / static data
Red-Black Tree	Color-based rules	General purpose (std::map)
Splay	Move used nodes to root	Caching systems
B-Tree	Multi-key nodes	Databases / Disk storage
		Simple

IMPORTANT NOTE

- There is a difference between
  - Traversal and Searching
- In-order, Pre-order, Post-order are traversal techniques not an efficient searching technique

NEXT

- We will study next AVL trees and Red and Black Trees in great Detail
- Please do not skip the class on Eid Account:
- These concepts are very Practical, useful and foundational for your Data Handling